

BinX Tech

Backend .NET Internship

Cardiac Patient Monitoring System

API Documentation

REST API • JWT Authentication • Role-Based Authorization

Patients • Doctors • Medications • Appointments • Vital Signs

Base URL

```
http://localhost:5180/api
```

In Postman, the collection uses:

```
{{baseUrl}}
```

The API provides CRUD operations for Patients, Doctors, Medications, Patient Medications, Appointments, and Vital Signs. All request and response DTOs are JSON-based.

Section 2 • Patients

Get All Patients

```
GET {{baseUrl}}/patients
```

Success: 200 OK

Returns all patients.

Get Patient By ID

```
GET {{baseUrl}}/patients/{id}
```

Success: 200 OK

Not found: 404 Not Found

Create Patient

```
POST {{baseUrl}}/patients
```

Success: 201 Created

Example request:

```
{
  "medicalRecordNumber": "MRN-0002",
  "firstName": "Khaled",
  "lastName": "Ahmad",
  "dateOfBirth": "1988-10-20",
  "gender": "Male",
  "phone": "0599444444",
  "email": "khaled.ahmad@example.com",
  "address": "Hebron, Palestine",
  "emergencyContactName": "Ali Ahmad",
  "emergencyContactPhone": "0599555555",
  "medicalNotes": "No known allergies"
}
```

Update Patient

```
PUT {{baseUrl}}/patients/{id}
```

Success: 200 OK

Not found: 404 Not Found

Delete Patient

```
DELETE {{baseUrl}}/patients/{id}
```

Success: 204 No Content

Not found: 404 Not Found

Section 3 • Doctors

Get All Doctors

```
GET {{baseUrl}}/doctors
```

Success: 200 OK

Get Doctor By ID

```
GET {{baseUrl}}/doctors/{id}
```

Success: 200 OK

Not found: 404 Not Found

Create Doctor

```
POST {{baseUrl}}/doctors
```

Success: 201 Created

Duplicate license number: 409 Conflict

Example request:

```
{
  "firstName": "Omar",
  "lastName": "Hassan",
  "email": "omar.hassan@example.com",
  "phone": "0599222222",
  "specialization": "Cardiology",
  "licenseNumber": "LIC-0001"
}
```

Update Doctor

```
PUT {{baseUrl}}/doctors/{id}
```

Success: 200 OK

Not found: 404 Not Found
Duplicate license number: 409 Conflict

Delete Doctor

```
DELETE {{baseUrl}}/doctors/{id}
```

Success: 204 No Content

Not found: 404 Not Found

Section 4 • Medications

Get All Medications

```
GET {{baseUrl}}/medications
```

Success: 200 OK

Get Medication By ID

```
GET {{baseUrl}}/medications/{id}
```

Success: 200 OK

Not found: 404 Not Found

Create Medication

```
POST {{baseUrl}}/medications
```

Success: 201 Created

Example request:

```
{
  "name": "Aspirin",
  "genericName": "Acetylsalicylic Acid",
  "description": "Used as an antiplatelet medication.",
  "strength": "100 mg",
  "dosageForm": "Tablet",
  "manufacturer": "Bayer"
}
```

Update Medication

```
PUT {{baseUrl}}/medications/{id}
```

Success: 200 OK

Not found: 404 Not Found

Delete Medication

```
DELETE {{baseUrl}}/medications/{id}
```

Success: 204 No Content

Not found: 404 Not Found

Section 5 • Patient Medications

Patient Medications represent the relationship between a patient and a medication, including dosage and treatment details.

Get All Patient Medications

```
GET {{baseUrl}}/patientmedications
```

Success: 200 OK

Get Patient Medication By ID

```
GET {{baseUrl}}/patientmedications/{id}
```

Success: 200 OK

Not found: 404 Not Found

Create Patient Medication

```
POST {{baseUrl}}/patientmedications
```

Success: 201 Created

Invalid patient or medication: 400 Bad Request

PatientId and MedicationId must reference existing records.

Example request:

```
{
  "patientId": 2,
  "medicationId": 2,
  "dosage": "50 mg",
  "frequency": "Twice daily",
  "route": "Oral",
  "startDate": "2026-08-10",
  "endDate": null,
  "status": "Active",
  "notes": "Take after meals"
}
```

Update Patient Medication

```
PUT {{baseUrl}}/patientmedications/{id}
```

Success: 200 OK

Not found: 404 Not Found

Delete Patient Medication

```
DELETE {{baseUrl}}/patientmedications/{id}
```

Success: 204 No Content

Not found: 404 Not Found

Section 6 • Appointments

Appointments connect an existing patient with an existing doctor.

Get All Appointments

```
GET {{baseUrl}}/appointments
```

Success: 200 OK

Get Appointment By ID

```
GET {{baseUrl}}/appointments/{id}
```

Success: 200 OK

Not found: 404 Not Found

Create Appointment

```
POST {{baseUrl}}/appointments
```

Success: 201 Created

Invalid patient or doctor: 400 Bad Request

PatientId and DoctorId must reference existing records.

Example request:

```
{
  "patientId": 2,
  "doctorId": 2,
  "appointmentDate": "2026-08-15T10:00:00Z",
  "appointmentType": "Cardiology Consultation",
  "status": "Scheduled",
  "reason": "Routine cardiac follow-up",
  "notes": "Bring previous medical reports"
}
```

Update Appointment

```
PUT {{baseUrl}}/appointments/{id}
```

Success: 200 OK

Not found: 404 Not Found

Delete Appointment

```
DELETE {{baseUrl}}/appointments/{id}
```

Success: 204 No Content

Not found: 404 Not Found

Section 7 • Vital Signs

Vital Signs are recorded measurements associated with a patient.

There is no update endpoint for Vital Signs in the current API design. A new measurement should be recorded instead of modifying an existing historical measurement.

Get All Vital Signs

```
GET {{baseUrl}}/vitalsigns
```

Success: 200 OK

Get Vital Sign By ID

GET {{baseUrl}}/vitalsigns/{id}

Success: 200 OK

Not found: 404 Not Found

Create Vital Sign

POST {{baseUrl}}/vitalsigns

Success: 201 Created

Invalid patient: 400 Bad Request

PatientId must reference an existing patient.

Example request:

```
{
  "patientId": 2,
  "heartRate": 72,
  "systolicBloodPressure": 120,
  "diastolicBloodPressure": 80,
  "respiratoryRate": 16,
  "temperature": 36.8,
  "oxygenSaturation": 98,
  "weight": 75.5,
  "recordedAt": "2026-08-10T17:30:00Z",
  "notes": "Normal vital signs"
}
```

Delete Vital Sign

DELETE {{baseUrl}}/vitalsigns/{id}

Success: 204 No Content

Not found: 404 Not Found

Section 8 • Endpoint Summary

Resource	GET All	GET By ID	POST	PUT	DELETE
Patients	✓	✓	✓	✓	✓
Doctors	✓	✓	✓	✓	✓
Medications	✓	✓	✓	✓	✓
Patient Medications	✓	✓	✓	✓	✓
Appointments	✓	✓	✓	✓	✓
Vital Signs	✓	✓	✓	—	✓

Section 9 • Postman

The accompanying Postman collection is organized into folders by resource and uses the baseUrl collection variable.

Import:

Cardiac-Patient-Monitoring-System.postman_collection.json

Then set baseUrl to the URL of the running API if the local port changes.